

# Ejercicios del Tema 6b

## Funciones con otros tipos de datos

### 1. Cambio de espacios por guión bajo

Escribe un programa que, dada una frase introducida por el usuario, cambie los espacios en blanco por un guión bajo `_`. Debe contar cuántos cambios se han producido. La petición de la frase debe programarse en la función `main`, los cambios y la cuenta en la función `espacio_por_guion`, y la frase definitiva y el número de cambios deben mostrarse en el terminal desde la función `main`. El ejercicio debe resolverse empleando una función con el siguiente prototipo:

```
int espacio_por_guion(char texto[]);
```

### 2. Cuenta letras

Escribe un programa que analice la frase del ejercicio anterior contando el número de veces que aparece cada vocal. Al final del programa, se debe pintar en el terminal, para cada vocal, una línea que contiene el mismo número de asteriscos que veces aparece esa vocal. Este programa debe estar programado empleando dos funciones: `cuentaLetra` y `pintaAsteriscos`. Pida al usuario la frase a modificar en la función `main`.

### 3. Audio

Las muestras obtenidas de la digitalización de una señal de audio (con un micrófono) están definidas por: una cadena de caracteres que representa el nombre de la señal, un número entero que es el número de muestras, y el resto de filas son los números reales que representan las muestras léídas por el sensor. Elabore un programa que:

- Use la estructura indicada más adelante para guardar una señal.
- Declare una variable de ese tipo de estructura e inicialice sus valores con 10 muestras. La inicialización se hace en el código, no la hace el usuario.

Definición de la estructura:

```
typedef struct
{
    char nombre[50];          // Nombre de la señal
    int n_muestras;           // Número de muestras
    float muestras[100];      // Muestras
}audio;
```

Finalmente, implemente una función que calcule y devuelva la mediana de ese vector.

## 4. Nóminas

Se debe calcular la nómina mensual de un empleado que trabaja por horas, el pago de cada hora trabajada depende de su categoría:

| categoría | pago x hora (€.) |
|-----------|------------------|
| A         | 30.00            |
| B         | 20.50            |
| C         | 18.50            |

Además, si el empleado trabaja más de 150 horas mensuales tiene una bonificación del 5 % de nómina. El programa que calcula la nómina está compuesto por dos funciones, una que solicita los datos del empleado por teclado y otra función que calcula la nómina mensual. El programa principal debe mostrar la nómina mensual calculada del empleado introducido.

Para los datos del empleado se debe usar la siguiente estructura:

```
typedef struct
{
    char nom[40], categoria;
    int horas;
    float nomina, pHora, bonf;
} empleado;
```

Los prototipos de las funciones que se tienen que utilizar son los siguientes:

```
empleado pedirDatos();
float calcularNomina (empleado emp);
```

### Ejemplo de ejecución:

Introduzca nombre empleado: Pepe Campo

Introduzca categoría del empleado: B

Introduzca horas trabajadas: 25

La nómina total del empleado Pepe Campo es: 512.50 euros, con un total de 0 euros de bonificación extra.

## 5. Texto

Realice un programa en lenguaje C que solicite una frase al usuario, la modifique y presente en pantalla el resultado, con las siguientes características:

- Tendrá una función, `int modificaT(char texto[])`, que realizará las siguientes acciones:
  - Solicitar al usuario una letra.
  - Solicitar al usuario una frase
  - Localizar la letra en el texto y sustituirla por: el carácter 5 si la letra aparece en mayúsculas, o por el carácter 7 si la letra aparece en minúsculas.
- El retorno de la función será el número de sustituciones realizadas.
- La impresión en el terminar de la frase modificada se hará desde la función `main` e incluirá el número de sustituciones realizadas. Si no se han realizado modificaciones, solo aparecerá la frase.
- Finalmente se preguntará al usuario si desea repetir el proceso con una nueva frase y se procederá en consecuencia.

## 6. Almacén

Se desea hacer un programa informático que calcule la cantidad de dinero inmovilizado en un almacén de productos. El dinero inmovilizado equivale al precio por unidad del producto multiplicado por la cantidad de productos disponibles en el almacén. Además, si la cantidad de productos en el almacén es menor o igual a 10 unidades, debe indicar la necesidad de hacer el pedido de dicho producto.

Estructure su código de la siguiente manera.

Generación del Almacén: vector de estructuras cuyo tamaño se establece al inicio del programa con una constante.

Formato del tipo de dato:

```
typedef struct
{
    char cod[5]; //Código del producto
    int stock;   //cantidad de producto en almacen
    float precio; //precio por unidad
}INV;
```

Funciones a utilizar en el programa:

Inicialización del Almacén: se realiza mediante la función `generaInventario()`. Esta función recibe como parámetros el vector de estructuras y la cantidad de elementos del vector. El usuario debe introducir por teclado cada uno de los elementos de la estructura a fin de inicializarla:

```
void generaInventario(INV inventario[], int numero_items);
```

Gestión del Almacén:: se realiza mediante una función que recibe como argumento un elemento (solo UNO) del vector de estructura previamente inicializado y calcula la cantidad de dinero inmovilizado en el almacén, en euros. Además, avisa al usuario si se necesita hacer un pedido cuando el stock sea menor o igual a 10. La función retorna la cantidad de euros inmovilizados del producto:

```
float calculaInmovilizado(INV producto)
```